Build a Reusable Data Fetching Hook (`useFetch`)

Tech stack: React 18 (Vite, JavaScript) · **Time limit:** 60 minutes (default, none given)

Problem: Build a reusable hook that any component can use to fetch data. It must expose loading / error / data states, cancel requests that are no longer needed, never show a stale (out-of-order) response, cache results so the same data is not fetched twice, and support refetch and retry. Demo it with a small paginated user list + detail panel.

PART A: Thought process

The general steps for any machine coding task

1. **Understand the requirements:** functional + non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions:** every action and its expected output. This becomes your checklist.
3. **Design the architecture:** modules and their one-line responsibilities.
4. **Design the state:** the minimum state, who owns it, what is derived.
5. **Design the contracts:** data shapes and function signatures.
6. **Trace one flow end to end** before coding.
7. **Name the hard parts** and decide the solution for each up front.
8. **Build in layers:** simplest working version first, then one layer at a time, always runnable.
9. **Cover every state:** loading, empty, error, success, plus edge cases.
10. **Wrap up:** trade-offs and what you would add with more time.

Time budget for 60 minutes

Minutes	Activity
0 - 7	Requirements, clarifying questions, assumptions
7 - 15	API of the hook, state shape, cache design, hard parts
15 - 25	Basic `useFetch` (status + data + error) with a mock API
25 - 35	Abort + race-condition safety
35 - 45	Shared cache + dedupe + `staleTime`
45 - 52	Retry with backoff, `refetch`, `mutate`, `keepPreviousData`
52 - 60	Demo UI states, edge cases, wrap-up

Think out loud

Say every decision and its reason as you make it ("I'm keying the cache by URL so two components asking for the same thing share one request"). The interviewer grades your reasoning as much as your code. If you run out of time, a clearly explained design for the parts you didn't finish still scores.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- `useFetch(key, fetcher, options)` returns `{ data, error, status, isLoading, isFetching, isError, refetch, mutate }`.
- Status lifecycle: `idle -> loading -> success | error`.
- Re-fetches automatically when the `key` (usually the URL) changes.
- Cancels the request when the component unmounts or the key changes (no wasted work, no "set state on unmounted component").
- Never shows an older response over a newer one (race conditions while typing / paginating).
- Caches responses by key; two components using the same key share **one** request and **one** cache entry.
- `staleTime`: cached data younger than this is used without refetching.
- `retry` with exponential backoff on failure.
- `refetch()` manually; `mutate(data | fn)` to update the cache locally.
- `enabled` / `key = null` to skip fetching (dependent queries).
- `keepPreviousData` for pagination (no spinner flash between pages).

Non-functional requirements

Area	What it means here
Performance	No duplicate requests; cache hits render instantly; only components using a key re-render when it changes
Scalability	Works with many components and many keys; store is outside React so it isn't tied to one tree
Error handling	Errors are state, not thrown; retries; aborted requests are not errors
Accessibility / usability	`aria-busy` on loading regions, `role="alert"` on errors, keyboard-usable buttons, skeletons instead of layout jumps
Reusability	Fetcher is injected: works with `fetch`, axios, GraphQL, or a mock
Extensibility	Cache module is separate from the hook, so invalidation, focus refetch or persistence can be added later

Clarifying questions to ask

1. Should the cache be shared across components, or is per-component state enough?
2. Do we need cancellation of in-flight requests, or only ignoring stale results?
3. Should failed requests retry automatically? How many times?
4. Is it read-only (GET), or do we also need mutations / optimistic updates?
5. Do we need SSR, Suspense, or refetch-on-window-focus?
6. Can I use a library (React Query / SWR), or must it be hand-written?

Assumptions we build with

- Hand-written, no libraries. React 18 (`useSyncExternalStore` available).
- In-memory cache shared across the app, keyed by a string (the URL).
- Default `retry = 2` , `retryDelay = 500ms` doubling each attempt; `staleTime = 0` (always revalidate, but show cached data meanwhile).
- `mutate` is a local cache update only; real server mutations are out of scope.
- No SSR, no Suspense, no persistence. A mock API with random latency and a failure-rate switch replaces the backend.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Component mounts with a key	`status: loading` , skeleton shown, one request started
Response arrives	`status: success` , data rendered
Request fails	Retries with backoff; after the last attempt `status: error` , error + Retry button
Click Retry / Refetch	New request; old data stays visible while refetching (`isFetching`)
Type "s", "sa", "sar", "sara" fast	Previous requests aborted; only "sara" results are shown, whatever order responses arrive in
Go back to a key fetched < `staleTime` ago	Rendered instantly from cache, no request
Two components use the same key	One network request, both update together
Component unmounts mid-request	Request aborted, unless another component still uses that key
Click Next page	Previous page stays visible (dimmed) until the new page arrives
Next page fails	Error with Retry / Back (not the old page pretending to be the new one)
`key = null` / `enabled: false`	Nothing fetched, `status: idle`
`mutate(fn)`	Cache entry updated; every component using that key re-renders

3. Design the architecture

Module	Responsibility
`lib/queryCache.js`	Framework-agnostic store: entries per key, in-flight dedupe, abort, retry, subscribers, ref counts
`hooks/useFetch.js`	React binding: subscribes to a key, triggers fetch on mount / key change, releases on unmount
`api/mockApi.js`	Fake backend: random latency, failure rate, `AbortSignal` support, request counters
`components/UserList.jsx`	Paginated + searchable list (race + cache + keepPreviousData demo)

Module	Responsibility
`components/UserDetail.jsx`	Dependent query by id, refetch, mutate
`components/SelectedBadge.jsx`	Same key as detail, proves dedupe + shared updates
`App.jsx`	Page / query / selection state, failure-rate switch, stats

```
App
├── SelectedBadge ─┐
├── UserList       ├── useFetch ──► queryCache (module singleton) ──► fetcher (mockApi / fetch)
└── UserDetail ────┘        ▲                │
                            └── subscribe ◄──┘   (notifies only listeners of the changed key)
```

Start simple: the first version is a hook with `useState` + `useEffect`. The cache is split out only when "two components, same key" becomes a requirement: that is the moment per-component state stops being enough.

4. Design the state / data model

Per cache entry (one per key)

State or field	Purpose
`status`	`'idle' | 'loading' | 'success' | 'error'`
`data`	Last successful response (kept during refetch and after a failed refetch)
`error`	Last error, `null` after success
`updatedAt`	Timestamp of last success, drives `staleTime`
`isFetching`	A request is in flight (true for first load and background refresh)

Store internals (not React state)

Map	Purpose
`entries`	key → immutable entry snapshot
`inflight`	key → `{ promise, controller }` for dedupe and abort
`listeners`	key → set of subscriber callbacks
`refCounts`	key → number of mounted hooks using it (abort when it hits 0)

```
export const EMPTY = Object.freeze({ status: 'idle', data: undefined, error: null, updatedAt: 0,
isFetching: false });
const entries = new Map();
const inflight = new Map();
const listeners = new Map();
const refCounts = new Map();
```

Senior note

- **Derived, not stored:** `isLoading = status === 'loading'`, `isError`, `isPreviousData`, "is stale" (`now - updatedAt >= staleTime`). Storing them would let them drift out of sync.
- **Who owns what:** server data lives in the cache, not in components. UI state (page, query, selected id) stays in `App`. The hook owns nothing but a ref to the latest fetcher and the last shown data.
- Entries are **replaced, never mutated**, so `useSyncExternalStore` can compare snapshots by reference.

5. Design the data structures and contracts

Example data

```
// User (stable numeric id: React keys, cache keys, selection all rely on it)
{ id: 2, name: 'Diya A.', email: 'diya2@example.com', role: 'Editor' }

// GET /users?page=1&q=sa
{ items: [/* 6 users */], page: 1, total: 3, totalPages: 1 }

// GET /users/2
{ id: 2, name: 'Diya A.', email: 'diya2@example.com', role: 'Editor' }
```

Stable ids matter because the cache key is built from them (`/users/2`) and list rows use them as React keys; an index would break when search filters the list.

Hook contract

```
const {
  data, error, status,           // 'idle' | 'loading' | 'success' | 'error'
  isLoading, isFetching, isError, isPreviousData, updatedAt,
  refetch,                        // () => Promise
  mutate,                         // (data | prev => next) => void
} = useFetch(
  key,                            // string | null  (null = don't fetch)
  fetcher,                        // (key, { signal }) => Promise<data>
  { enabled: true, staleTime: 0, retry: 2, retryDelay: 500, keepPreviousData: false },
);
```

Fetcher contract

The fetcher receives the key and an `AbortSignal`; it must reject when aborted. Real usage:

```
const fetcher = (url, { signal }) =>
  fetch(url, { signal }).then((r) => { if (!r.ok) throw new Error(`HTTP ${r.status}`); return
r.json(); });
```

6. Trace the data flow

User types "sa" in the search box:

```
input onChange → setQuery('sa'), setPage(1)
        ↓
```

```
UserList re-renders → key = '/users?page=1&q=sa'
        ↓
useFetch effect cleanup for old key '/users?page=1&q=s'
   → release() → refCount 0 → cancelQuery → controller.abort(), inflight entry removed
        ↓
useFetch effect for new key → retain(key) → isStale? yes → fetchQuery(key)
   → no inflight for key → new AbortController → setEntry(key, {status:'loading', isFetching:true})
   → listeners of THIS key notified → useSyncExternalStore re-renders UserList
   → keepPreviousData: shows old rows dimmed + "Loading page…"
        ↓
fetcher(key, { signal }) → mock resolves after 300-1200ms
        ↓
isCurrent()? (inflight.get(key) === this record) → yes
   → setEntry(key, {status:'success', data, updatedAt: now, isFetching:false})
        ↓
UserList re-renders with the new rows
(the aborted "s" request rejects with AbortError → isCurrent() false → writes nothing)
```

7. Important technical challenges

Challenge 1: Race conditions (stale responses)

Problem: User types "s" then "sa". The "s" request takes 1200ms, the "sa" request 300ms. A naive `useEffect(() => { fetch(url).then(setData) }, [url])` sets "sa" results, then 900ms later overwrites them with "s" results. The screen now lies.

Solution: two layers.

1. Abort the old request when the key changes (saves bandwidth).
2. A late response only writes if it is still *the* request for that key (`isCurrent()`), and it writes to its own key's entry, so it can never land on another key's data.

```
const record = { controller: new AbortController() };
const isCurrent = () => inflight.get(key) === record;
record.promise = withRetry(/* ... */).then((data) => {
  if (isCurrent()) setEntry(key, { status: 'success', data, error: null, updatedAt: Date.now(),
isFetching: false });
});

export function cancelQuery(key) {
  const record = inflight.get(key);
  if (!record) return;
  inflight.delete(key);          // detach FIRST: the aborted promise can no longer write
  record.controller.abort();
}
```

Challenge 2: Shared cache + request dedupe across components

Problem: `UserDetail` and `SelectedBadge` both need `/users/2` . With per-component state that is two requests, two copies of the data, and `mutate` in one doesn't update the other.

Solution: a module-level store keyed by URL. `fetchQuery` returns the existing in-flight promise if there is one. Components subscribe with `useSyncExternalStore` , which is tear-free under concurrent rendering and only re-renders subscribers of the changed key.

```
export function fetchQuery(key, fetcher, opts) {
  const existing = inflight.get(key);
  if (existing) return existing.promise;   // dedupe
  // ... start a new request
}

// in the hook
const sub = useCallback((cb) => (key == null ? NOOP : subscribe(key, cb)), [key]);
const entry = useSyncExternalStore(sub, () => (key == null ? EMPTY : getEntry(key)));
```

Challenge 3: When is it safe to abort a shared request?

Problem: If `UserDetail` unmounts and aborts `/users/2`, the badge (still mounted, same key) is left loading forever.

Solution: reference counting. Each mounted hook `retain`s its key; the cleanup `release`s it; only the last release aborts.

```
export function retain(key) {
  refCounts.set(key, (refCounts.get(key) ?? 0) + 1);
  return () => {
    const n = refCounts.get(key) - 1;
    if (n > 0) return refCounts.set(key, n);
    refCounts.delete(key);
    cancelQuery(key);
  };
}

useEffect(() => {
  if (!active) return;
  const release = retain(key);
  if (isStale(key, staleTime)) fetchQuery(key, fetcherRef.current, retryRef.current);
  return release;
}, [key, active, staleTime]);
```

Challenge 4: Unstable dependencies + retry that respects abort

Problem: Callers pass inline arrows (`useFetch(url, (u, o) => fetch(u, o))`). Putting `fetcher` in effect deps refetches on every render, an infinite loop if the fetch sets state. Separately, a retry loop that sleeps 2s must stop the moment the request is aborted.

Solution: keep the latest fetcher in a ref (deps are only `key`, `enabled`, `staleTime`), and make `sleep` abortable.

```
const fetcherRef = useRef(fetcher);
useEffect(() => { fetcherRef.current = fetcher; });

export async function withRetry(fn, { retry = 0, retryDelay = 500, signal } = {}) {
  for (let attempt = 0; ; attempt++) {
    try { return await fn(signal); }
    catch (err) {
      if (signal?.aborted || attempt >= retry) throw err;
      await sleep(retryDelay * 2 ** attempt, signal);   // 500, 1000, 2000 … rejects on abort
    }
```

```
  }
}
```

8. Implement in the correct order

#	Step	Working result
1	Mock API with random latency + `signal` support	Can call `fakeFetch('/users?page=1')`
2	Naive `useFetch` with `useState` + `useEffect`	List renders with loading / error / data
3	Add `AbortController` in effect cleanup	Typing fast aborts old requests
4	Move state into `queryCache` + `useSyncExternalStore`	Same behavior, store outside React
5	In-flight dedupe + `isCurrent()` guard	Two components, one request; no stale overwrite
6	`retain` / `release` ref counting	Unmount aborts only when nobody else needs the key
7	`staleTime` + `isFetching` (stale-while-revalidate)	Going back to a page is instant
8	`withRetry` with abortable backoff	Flaky API recovers; abort stops retries
9	`refetch` , `mutate` , `key = null` / `enabled`	Detail panel, favourite toggle, dependent query
10	`keepPreviousData`	Pagination without spinner flash
11	Demo UI states, a11y, failure-rate switch, stats	Every row of the use-case table is visible
12	Node assert self-check of `queryCache`	Dedupe, retry, abort, race verified

9. Cover all states and edge cases

State / edge case	How it is handled
Initial (no key)	`status: idle` , "Select a user" message, nothing fetched
Loading (first load)	Skeleton rows with `aria-busy="true"`
Background refetch	Data stays visible, "Refreshing…" pill (`isFetching && data`)
Success	Rendered list / detail
Empty	"No users match …"
Error, no data	`role="alert"` message + Retry
Error, has cached data	Cached data kept, "Refresh failed, showing cached data"
Error on a new page with `keepPreviousData`	Error + Retry / Back, not the old page
Out-of-order responses	Aborted + `isCurrent()` guard; late write ignored
Unmount mid-request	Aborted if last user; no state update after unmount

State / edge case	How it is handled
Same key in 2 components	One request; `mutate` updates both
Key flips A → B → A quickly	A's first request was detached on cancel, so the second A request is the only one that can write
Abort during retry backoff	`sleep` rejects immediately, no more attempts
Fetcher throws synchronously	`withRetry` is `async`, so it becomes a rejection
StrictMode double effect (dev)	First request aborted, second wins (correct, just 2 requests in dev)
Inline fetcher each render	Stored in a ref, not in deps: no refetch loop
Next clicked while page still loading	Next disabled while `isPreviousData`

10. Final structure

```
_97_Build_a_Reusable_Data_Fetching_Hook/
├── index.html
├── package.json
├── vite.config.js
├── check.mjs                    # node assert self-check of queryCache
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── App.css
    ├── lib/
    │   └── queryCache.js        # store: entries, inflight, refCounts, retry, abort
    ├── hooks/
    │   └── useFetch.js          # React binding
    ├── api/
    │   └── mockApi.js           # fake backend
    └── components/
        ├── UserList.jsx
        ├── UserDetail.jsx
        └── SelectedBadge.jsx
```

11. Full working code

Run with `npm install` then `npm run dev`. `node check.mjs` runs the self-check.

package.json

```
{
  "name": "use-fetch-hook",
  "private": true,
  "type": "module",
  "scripts": { "dev": "vite", "build": "vite build", "preview": "vite preview" },
  "dependencies": { "react": "18.3.1", "react-dom": "18.3.1" },
  "devDependencies": { "@vitejs/plugin-react": "4.3.4", "vite": "5.4.11" }
}
```

src/lib/queryCache.js

```
// Module-level store shared by every useFetch call: one entry + at most one in-flight request per
key.
export const EMPTY = Object.freeze({ status: 'idle', data: undefined, error: null, updatedAt: 0,
isFetching: false });

const entries = new Map(); // key -> immutable entry snapshot (new object on every change)
const inflight = new Map(); // key -> { promise, controller }
const listeners = new Map(); // key -> Set<() => void>
const refCounts = new Map(); // key -> number of mounted hooks using the key

export const getEntry = (key) => entries.get(key) ?? EMPTY;

const settledStatus = (e) => (e.error ? 'error' : e.data !== undefined ? 'success' : 'idle');

export function setEntry(key, patch) {
  entries.set(key, { ...getEntry(key), ...patch });
  listeners.get(key)?.forEach((fn) => fn());
}

export function subscribe(key, fn) {
  if (!listeners.has(key)) listeners.set(key, new Set());
  listeners.get(key).add(fn);
  return () => listeners.get(key).delete(fn);
}

export function isStale(key, staleTime) {
  const { updatedAt } = getEntry(key);
  return !updatedAt || Date.now() - updatedAt >= staleTime;
}

export function sleep(ms, signal) {
  return new Promise((resolve, reject) => {
    if (signal?.aborted) return reject(signal.reason);
    const t = setTimeout(resolve, ms);
    signal?.addEventListener('abort', () => { clearTimeout(t); reject(signal.reason); }, { once: true
});
  });
}

// Exponential backoff: retryDelay, 2x, 4x ... Stops immediately once aborted.
export async function withRetry(fn, { retry = 0, retryDelay = 500, signal } = {}) {
  for (let attempt = 0; ; attempt++) {
    try {
      return await fn(signal);
    } catch (err) {
      if (signal?.aborted || attempt >= retry) throw err;
      await sleep(retryDelay * 2 ** attempt, signal);
    }
  }
}

export function fetchQuery(key, fetcher, { retry = 0, retryDelay = 500 } = {}) {
  const existing = inflight.get(key);
  if (existing) return existing.promise; // dedupe: same key already loading -> share it

  const record = { controller: new AbortController() };
  // A late response must only write if it is still THE request for this key (not cancelled /
replaced).
  const isCurrent = () => inflight.get(key) === record;
```

```
  const prev = getEntry(key);
  setEntry(key, { isFetching: true, status: prev.data === undefined ? 'loading' : prev.status });

  record.promise = withRetry((signal) => fetcher(key, { signal }), { retry, retryDelay, signal:
record.controller.signal })
    .then(
      (data) => {
        if (isCurrent()) setEntry(key, { status: 'success', data, error: null, updatedAt: Date.now(),
isFetching: false });
        return data;
      },
      (error) => {
        if (isCurrent()) setEntry(key, { status: 'error', error, isFetching: false });
      },
    )
    .finally(() => {
      if (isCurrent()) inflight.delete(key);
    });

  inflight.set(key, record);
  return record.promise;
}

export function cancelQuery(key) {
  const record = inflight.get(key);
  if (!record) return;
  inflight.delete(key); // detach first so the aborted promise can never write
  record.controller.abort();
  const e = getEntry(key);
  setEntry(key, { isFetching: false, status: settledStatus(e) });
}

// Each mounted hook retains its key; when the last one lets go, the request is aborted.
export function retain(key) {
  refCounts.set(key, (refCounts.get(key) ?? 0) + 1);
  return () => {
    const n = refCounts.get(key) - 1;
    if (n > 0) return refCounts.set(key, n);
    refCounts.delete(key);
    cancelQuery(key);
  };
}

export function setQueryData(key, updater) {
  const data = typeof updater === 'function' ? updater(getEntry(key).data) : updater;
  setEntry(key, { data, status: 'success', error: null, updatedAt: Date.now() });
}

export function clearCache() {
  inflight.forEach((r) => r.controller.abort());
  [entries, inflight, refCounts].forEach((m) => m.clear());
}
```

src/hooks/useFetch.js

```
import { useCallback, useEffect, useRef, useSyncExternalStore } from 'react';
import { EMPTY, fetchQuery, getEntry, isStale, retain, setQueryData, subscribe } from
'../lib/queryCache';
```

```
const NOOP = () => {};

/**
 * useFetch(key, fetcher, options)
 * key      string | null  - cache key (usually the URL). null = don't fetch (dependent queries).
 * fetcher  (key, { signal }) => Promise<data>
 * options  { enabled = true, staleTime = 0, retry = 2, retryDelay = 500, keepPreviousData = false }
 */
export function useFetch(key, fetcher, options = {}) {
  const { enabled = true, staleTime = 0, retry = 2, retryDelay = 500, keepPreviousData = false } =
options;
  const active = key != null && enabled;

  // Latest fetcher/options without putting them in effect deps (inline arrows change every render).
  const fetcherRef = useRef(fetcher);
  const retryRef = useRef({ retry, retryDelay });
  useEffect(() => {
    fetcherRef.current = fetcher;
    retryRef.current = { retry, retryDelay };
  });

  const sub = useCallback((cb) => (key == null ? NOOP : subscribe(key, cb)), [key]);
  const entry = useSyncExternalStore(sub, () => (key == null ? EMPTY : getEntry(key)));

  useEffect(() => {
    if (!active) return;
    const release = retain(key);
    if (isStale(key, staleTime)) fetchQuery(key, fetcherRef.current, retryRef.current);
    return release; // unmount / key change -> abort if nobody else needs it
  }, [key, active, staleTime]);

  const refetch = useCallback(
    () => (key == null ? Promise.resolve() : fetchQuery(key, fetcherRef.current, retryRef.current)),
    [key],
  );
  const mutate = useCallback((updater) => key != null && setQueryData(key, updater), [key]);

  // Pagination: show the last page's data while the next key loads instead of flashing a spinner.
  const lastData = useRef(undefined);
  if (entry.data !== undefined) lastData.current = entry.data;
  const showPrevious = keepPreviousData && entry.data === undefined && lastData.current !== undefined;

  return {
    data: showPrevious ? lastData.current : entry.data,
    error: entry.error,
    status: entry.status,
    isLoading: entry.status === 'loading' && !showPrevious,
    isFetching: entry.isFetching,
    isError: entry.status === 'error',
    isPreviousData: showPrevious,
    updatedAt: entry.updatedAt,
    refetch,
    mutate,
  };
}
```

src/api/mockApi.js

```
// Fake backend: random latency (so responses arrive out of order), optional failure rate, abort
support.
```

```
const FIRST = ['Aarav', 'Diya', 'Kabir', 'Meera', 'Rohan', 'Sara', 'Vivaan', 'Anaya', 'Ishaan',
'Zoya', 'Arjun', 'Nisha'];
const ROLES = ['Admin', 'Editor', 'Viewer'];

const USERS = Array.from({ length: 36 }, (_, i) => ({
  id: i + 1,
  name: `${FIRST[i % 12]} ${String.fromCharCode(65 + Math.floor(i / 12))}.`,
  email: `${FIRST[i % 12].toLowerCase()}${i + 1}@example.com`,
  role: ROLES[i % 3],
}));

const PAGE_SIZE = 6;
export const apiSettings = { failRate: 0, minLatency: 300, maxLatency: 1200 };

// Tiny observable request counter for the demo UI.
let stats = { started: 0, aborted: 0, failed: 0 };
const statListeners = new Set();
const bump = (k) => {
  stats = { ...stats, [k]: stats[k] + 1 };
  statListeners.forEach((fn) => fn());
};
export const getStats = () => stats;
export const onStats = (fn) => (statListeners.add(fn), () => statListeners.delete(fn));

export const usersUrl = (page, q) => `/users?page=${page}&q=${encodeURIComponent(q)}`;
export const userUrl = (id) => `/users/${id}`;

function route(path) {
  const url = new URL(path, 'http://mock');
  const byId = url.pathname.match(/^\/users\/(\d+)$/);
  if (byId) {
    const user = USERS.find((u) => u.id === Number(byId[1]));
    if (!user) throw new Error('404: user not found');
    return { ...user };
  }
  if (url.pathname === '/users') {
    const q = (url.searchParams.get('q') ?? '').trim().toLowerCase();
    const page = Number(url.searchParams.get('page') ?? 1);
    const filtered = USERS.filter((u) => u.name.toLowerCase().includes(q));
    return {
      items: filtered.slice((page - 1) * PAGE_SIZE, page * PAGE_SIZE),
      page,
      total: filtered.length,
      totalPages: Math.max(1, Math.ceil(filtered.length / PAGE_SIZE)),
    };
  }
  throw new Error(`404: ${url.pathname}`);
}

export function fakeFetch(path, { signal } = {}) {
  return new Promise((resolve, reject) => {
    if (signal?.aborted) return reject(new DOMException('Aborted', 'AbortError'));
    bump('started');
    const { minLatency, maxLatency } = apiSettings;
    const timer = setTimeout(() => {
      try {
        if (Math.random() < apiSettings.failRate) throw new Error(`500: server error on ${path}`);
        resolve(route(path));
      } catch (err) {
        bump('failed');
        reject(err);
      }
```

```
    }, minLatency + Math.random() * (maxLatency - minLatency));
    signal?.addEventListener(
      'abort',
      () => {
        clearTimeout(timer);
        bump('aborted');
        reject(new DOMException('Aborted', 'AbortError'));
      },
      { once: true },
    );
  });
}
```

src/components/UserList.jsx

```
import { useFetch } from '../hooks/useFetch';
import { fakeFetch, usersUrl } from '../api/mockApi';

export default function UserList({ page, query, selectedId, onPage, onSelect }) {
  const { data, error, isLoading, isFetching, isError, isPreviousData, refetch } = useFetch(
    usersUrl(page, query),
    fakeFetch,
    { staleTime: 30_000, keepPreviousData: true },
  );

  if (isLoading) {
    return (
      <ul className="list" aria-busy="true" aria-label="Loading users">
        {Array.from({ length: 6 }, (_, i) => <li key={i} className="skeleton" />)}
      </ul>
    );
  }

  // No data for THIS key (previous page doesn't count) -> show the error, not the old page.
  if (isError && (!data || isPreviousData)) {
    return (
      <div className="state error" role="alert">
        <p>Could not load users: {error.message}</p>
        <button onClick={refetch}>Retry</button>{' '}
        {page > 1 && <button onClick={() => onPage(page - 1)}>Back</button>}
      </div>
    );
  }

  if (!data) return null;
  if (data.items.length === 0) return <p className="state">No users match “{query}”.</p>;

  return (
    <section aria-busy={isFetching}>
      <div className="list-meta">
        <span>{data.total} users</span>
        {isFetching && <span className="pill">{isPreviousData ? 'Loading page…' : 'Refreshing…'}
</span>}
        {isError && <span className="pill warn">Refresh failed, showing cached data</span>}
      </div>
      <ul className={`list ${isPreviousData ? 'dim' : ''}`}>
        {data.items.map((u) => (
          <li key={u.id}>
            <button className={`row ${u.id === selectedId ? 'active' : ''}`} onClick={() =>
onSelect(u.id)}>
```

```
              <strong>{u.name}</strong>
              <span>{u.role}</span>
            </button>
          </li>
        ))}
      </ul>
      <nav className="pager" aria-label="Pagination">
        <button disabled={page <= 1} onClick={() => onPage(page - 1)}>Prev</button>
        <span>Page {data.page} / {data.totalPages}</span>
        <button disabled={isPreviousData || page >= data.totalPages} onClick={() => onPage(page +
1)}>Next</button>
      </nav>
    </section>
  );
}
```

src/components/UserDetail.jsx

```
import { useFetch } from '../hooks/useFetch';
import { fakeFetch, userUrl } from '../api/mockApi';

export default function UserDetail({ id }) {
  // Dependent query: no id -> key null -> nothing fetched.
  const { data, error, isLoading, isFetching, isError, refetch, mutate, updatedAt } = useFetch(
    id ? userUrl(id) : null,
    fakeFetch,
    { staleTime: 10_000 },
  );

  if (!id) return <p className="state">Select a user to see details.</p>;
  if (isLoading) return <p className="state" aria-busy="true">Loading user #{id}…</p>;
  if (isError && !data) {
    return (
      <div className="state error" role="alert">
        <p>{error.message}</p>
        <button onClick={refetch}>Retry</button>
      </div>
    );
  }
  if (!data) return null;

  return (
    <article className="detail">
      <h2>
        {data.name} {data.favourite && <span aria-label="favourite">★</span>}
      </h2>
      <dl>
        <dt>Email</dt><dd>{data.email}</dd>
        <dt>Role</dt><dd>{data.role}</dd>
        <dt>Fetched</dt><dd>{new Date(updatedAt).toLocaleTimeString()}</dd>
      </dl>
      <div className="actions">
        <button onClick={refetch} disabled={isFetching}>{isFetching ? 'Refreshing…' : 'Refetch'}
</button>
        <button onClick={() => mutate((u) => ({ ...u, favourite: !u.favourite }))}>
          {data.favourite ? 'Unfavourite' : 'Favourite'} (local mutate)
        </button>
      </div>
      {isError && <p className="pill warn" role="alert">Refresh failed: {error.message}</p>}
    </article>
```

```
  );
}
```

src/components/SelectedBadge.jsx

```
import { useFetch } from '../hooks/useFetch';
import { fakeFetch, userUrl } from '../api/mockApi';

// Same key as UserDetail -> shares the cache entry and the in-flight request (one network call).
export default function SelectedBadge({ id }) {
  const { data, isLoading } = useFetch(id ? userUrl(id) : null, fakeFetch, { staleTime: 10_000 });
  if (!id) return <span className="badge muted">No selection</span>;
  return <span className="badge">{isLoading ? '…' : `${data?.name ?? '?'}${data?.favourite ? ' ★' :
''}`}</span>;
}
```

src/App.jsx

```
import { useState, useSyncExternalStore } from 'react';
import UserList from './components/UserList';
import UserDetail from './components/UserDetail';
import SelectedBadge from './components/SelectedBadge';
import { apiSettings, getStats, onStats } from './api/mockApi';
import './App.css';

export default function App() {
  const [query, setQuery] = useState('');
  const [page, setPage] = useState(1);
  const [selectedId, setSelectedId] = useState(null);
  const [showDetail, setShowDetail] = useState(true);
  const [failRate, setFailRate] = useState(apiSettings.failRate);
  const stats = useSyncExternalStore(onStats, getStats);

  const changeFailRate = (e) => {
    apiSettings.failRate = Number(e.target.value);
    setFailRate(apiSettings.failRate);
  };

  return (
    <div className="app">
      <header>
        <h1>useFetch demo</h1>
        <SelectedBadge id={selectedId} />
      </header>

      <div className="toolbar">
        <input
          type="search"
          placeholder="Search users…"
          aria-label="Search users"
          value={query}
          onChange={(e) => { setQuery(e.target.value); setPage(1); }}
        />
        <label>
          Failure rate{' '}
          <select value={failRate} onChange={changeFailRate}>
            <option value={0}>0%</option>
            <option value={0.5}>50%</option>
            <option value={1}>100%</option>
```

```
          </select>
        </label>
        <label>
          <input type="checkbox" checked={showDetail} onChange={(e) =>
setShowDetail(e.target.checked)} /> Detail panel
        </label>
        <output className="stats" aria-live="polite">
          requests {stats.started} · aborted {stats.aborted} · failed {stats.failed}
        </output>
      </div>

      <main>
        <UserList page={page} query={query} selectedId={selectedId} onPage={setPage} onSelect=
{setSelectedId} />
        {showDetail && <UserDetail id={selectedId} />}
      </main>
    </div>
  );
}
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

src/App.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: #f5f6f8; color: #1d2330; }
button { font: inherit; cursor: pointer; border: 1px solid #c8cdd6; background: #fff; border-radius:
6px; padding: 6px 12px; }
button:disabled { opacity: 0.5; cursor: not-allowed; }
button:focus-visible, input:focus-visible, select:focus-visible { outline: 2px solid #3b6ef5; outline-
offset: 2px; }

.app { max-width: 900px; margin: 0 auto; padding: 24px 16px; }
header { display: flex; justify-content: space-between; align-items: center; }
h1 { font-size: 22px; margin: 0; }
.badge { background: #e7edff; color: #2848b8; padding: 4px 10px; border-radius: 999px; font-size:
14px; }
.badge.muted { background: #eceef2; color: #667; }

.toolbar { display: flex; flex-wrap: wrap; gap: 12px; align-items: center; margin: 16px 0; }
.toolbar input[type='search'] { flex: 1 1 200px; padding: 8px 10px; border: 1px solid #c8cdd6; border-
radius: 6px; font: inherit; }
.stats { font-size: 13px; color: #556; font-variant-numeric: tabular-nums; }

main { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
@media (max-width: 640px) { main { grid-template-columns: 1fr; } }
main > * { background: #fff; border: 1px solid #e1e4ea; border-radius: 10px; padding: 16px; margin: 0;
}
```

```
.list { list-style: none; margin: 0; padding: 0; display: grid; gap: 6px; transition: opacity 0.2s; }
.list.dim { opacity: 0.55; }
.row { width: 100%; display: flex; justify-content: space-between; text-align: left; }
.row.active { border-color: #3b6ef5; background: #f0f4ff; }
.row span { color: #667; font-size: 14px; }
.skeleton { height: 36px; border-radius: 6px; background: linear-gradient(90deg, #eceef2, #f7f8fa,
#eceef2); background-size: 200% 100%; animation: shimmer 1.2s infinite; }
@keyframes shimmer { to { background-position: -200% 0; } }

.list-meta { display: flex; gap: 8px; align-items: center; margin-bottom: 10px; font-size: 14px;
color: #556; }
.pill { font-size: 12px; background: #eef1f6; padding: 2px 8px; border-radius: 999px; }
.pill.warn { background: #fff3d6; color: #8a5a00; }
.pager { display: flex; justify-content: space-between; align-items: center; margin-top: 12px; font-
size: 14px; }

.state { color: #556; }
.state.error { color: #a32020; }
.detail h2 { margin-top: 0; font-size: 20px; }
.detail dl { display: grid; grid-template-columns: auto 1fr; gap: 6px 12px; }
.detail dt { color: #667; }
.detail dd { margin: 0; }
.actions { display: flex; gap: 8px; flex-wrap: wrap; }
@media (prefers-reduced-motion: reduce) { .skeleton { animation: none; } .list { transition: none; } }
```

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>useFetch demo</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({ plugins: [react()] });
```

check.mjs (self-check)

```
import assert from 'node:assert/strict';
import { clearCache, fetchQuery, getEntry, retain, setQueryData, withRetry } from
'./src/lib/queryCache.js';

const deferred = () => { let resolve; const p = new Promise((r) => (resolve = r)); return { p, resolve
}; };

// 1. dedupe: two callers, one request
```

```
{
  clearCache();
  let calls = 0;
  const f = async () => { calls++; return 'A'; };
  await Promise.all([fetchQuery('k', f), fetchQuery('k', f)]);
  assert.equal(calls, 1);
  assert.equal(getEntry('k').data, 'A');
  assert.equal(getEntry('k').status, 'success');
}

// 2. retry succeeds on 3rd attempt
{
  clearCache();
  let calls = 0;
  await fetchQuery('r', async () => { if (++calls < 3) throw new Error('boom'); return 'ok'; }, {
retry: 2, retryDelay: 1 });
  assert.equal(calls, 3);
  assert.equal(getEntry('r').data, 'ok');
}

// 3. retries exhausted -> error status
{
  clearCache();
  let calls = 0;
  await fetchQuery('e', async () => { calls++; throw new Error('down'); }, { retry: 1, retryDelay: 1
});
  assert.equal(calls, 2);
  assert.equal(getEntry('e').status, 'error');
  assert.equal(getEntry('e').error.message, 'down');
}

// 4. last release aborts the request; aborted result never written
{
  clearCache();
  let seenSignal;
  const d = deferred();
  const release = retain('c');
  fetchQuery('c', (_k, { signal }) => { seenSignal = signal; return d.p; });
  assert.equal(getEntry('c').status, 'loading');
  release();
  assert.equal(seenSignal.aborted, true);
  assert.equal(getEntry('c').status, 'idle');
  d.resolve('late');
  await new Promise((r) => setTimeout(r, 5));
  assert.equal(getEntry('c').data, undefined);
}

// 5. race: cancelled slow request resolves AFTER the newer one -> newer data wins
{
  clearCache();
  const slow = deferred();
  const fast = deferred();
  const rel1 = retain('x');
  fetchQuery('x', () => slow.p);
  rel1(); // e.g. user navigated away and back
  const rel2 = retain('x');
  const second = fetchQuery('x', () => fast.p);
  fast.resolve('new');
  await second;
  slow.resolve('old');
  await new Promise((r) => setTimeout(r, 5));
  assert.equal(getEntry('x').data, 'new');
```

```
  rel2();
}

// 6. shared key: one hook unmounting does not abort the other's request
{
  clearCache();
  let signal;
  const d = deferred();
  const a = retain('s');
  const b = retain('s');
  const p = fetchQuery('s', (_k, o) => { signal = o.signal; return d.p; });
  a();
  assert.equal(signal.aborted, false);
  d.resolve('kept');
  await p;
  assert.equal(getEntry('s').data, 'kept');
  b();
}

// 7. mutate + abort stops retry backoff
{
  clearCache();
  setQueryData('m', { n: 1 });
  setQueryData('m', (u) => ({ ...u, n: u.n + 1 }));
  assert.equal(getEntry('m').data.n, 2);
  const ac = new AbortController();
  const t0 = Date.now();
  const p = withRetry(async () => { throw new Error('x'); }, { retry: 5, retryDelay: 1000, signal:
ac.signal });
  setTimeout(() => ac.abort(), 10);
  await assert.rejects(p);
  assert.ok(Date.now() - t0 < 500);
}

console.log('queryCache self-check: all 7 passed');
```

12. Wrap-up

Edge cases handled

Race conditions, abort on unmount / key change, shared-key ref counting, request dedupe, abortable retry backoff, stale-while-revalidate, failed background refresh keeping cached data, dependent queries (`key = null`), inline fetchers, StrictMode double effects, pagination without flashing, error on a new page with previous data.

Trade-offs to mention

- **Module singleton vs Context provider:** a singleton is simplest and works anywhere, but it is global (tests must `clearCache()` , SSR would leak between requests). A `<QueryClientProvider>` fixes both at the cost of setup.
- **Key = string:** simple and readable; object keys would need stable serialization.
- **No cache eviction:** entries live forever. Fine for a demo; production needs `gcTime` (drop entries with 0 subscribers after N minutes) or an LRU.
- **`mutate` is local only:** a refetch already in flight will overwrite it. Real optimistic updates need cancel-in-flight + rollback on server error.

- `keepPreviousData` **uses a ref written during render:** pragmatic and safe here (idempotent), but not "pure render" in the strictest sense.
- **Hand-written vs React Query / SWR:** in production I'd use the library; this shows I understand what it does underneath.

What to add with more time

- `invalidate(prefix)` after mutations, refetch on window focus / reconnect, polling interval.
- Garbage collection of unused entries, max cache size.
- `select` option to derive a slice of data and skip re-renders.
- Suspense mode (`throw promise`) and SSR hydration.
- Unit tests with Vitest + React Testing Library + fake timers; MSW for network mocks.
- Infinite queries (`useInfiniteFetch`) with cursor pagination.

13. Counter questions the interviewer may ask

Requirements

Q1. Why build this instead of using React Query or SWR?

In a real product I would use one of them. The interview tests whether I understand the problems they solve: races, dedupe, caching, retries, cancellation. Writing the core shows that, and it helps when debugging the library later.

Q2. What is the difference between `isLoading` and `isFetching`?

`isLoading` means there is no data yet and a request is running, so show a skeleton. `isFetching` is any request in flight, including a background refresh while old data is on screen, so show a small "Refreshing..." hint. Mixing them up causes a spinner flashing over good data.

State design

Q3. Why `useSyncExternalStore` instead of `useState` + a subscription in `useEffect`?

The store lives outside React. `useSyncExternalStore` reads it during render, so it can't "tear" (two components showing different versions in one concurrent render). It also avoids the window between first render and the effect subscribing, where an update could be missed.

Q4. Why are cache entries immutable?

`useSyncExternalStore` compares snapshots with `Object.is` . If I mutated the entry in place, the reference would be the same and React would skip the re-render. A new object per change makes "did it change?" a pointer comparison.

Q5. Why is the cache keyed by the URL string?

It is unique per resource and readable in devtools. Pages and searches get separate entries automatically. If keys were objects I'd need stable serialization (sorted keys), which is what React Query does with its array keys.

Challenge 1: Race conditions

Q6. Isn't aborting enough? Why also the `isCurrent()` check?

Abort is best-effort: a fetcher may ignore the signal, or the response may already be resolving when abort fires. The `isCurrent()` check is the guarantee: only the request currently registered for the key may write. Abort saves bandwidth; the check guarantees correctness.

Q7. What does the `ignore` flag pattern look like, and why not just use that?

```
useEffect(() => { let ignore = false; fetch(url).then((d) => !ignore && setData(d)); return () => {
ignore = true; }; }, [url]);
```

It fixes races for one component but doesn't cancel the network request and doesn't share anything across components. It is a good step 2, and I'd mention it as the minimal fix.

Challenge 2: Cache and dedupe

Q8. What if 50 components mount with the same key at once?

First call creates the in-flight record; the other 49 get the same promise from `inflight.get(key)`. One request, one entry, 50 subscribers notified once when it resolves.

Q9. What if the data is huge, or there are 1 million different keys?

Memory grows forever without eviction. I'd add garbage collection: when a key's ref count drops to 0, start a timer (`gcTime`, e.g. 5 min) and delete the entry if nobody re-subscribes. Or cap the cache with an LRU. For huge lists, paginate on the server and virtualize rendering.

Q10. How would you invalidate the list after updating a user?

Add `invalidate(predicate)`: mark matching entries stale (`updatedAt = 0`) and immediately refetch those with active subscribers (ref count > 0). Example: `invalidate((k) => k.startsWith('/users?'))` after a successful PATCH.

Challenge 3: Abort with shared keys

Q11. What happens in React StrictMode?

In dev, effects run mount → cleanup → mount. The first `retain` / `release` aborts the first request and the second mount starts a new one. Behavior stays correct; you just see one extra (aborted) request in dev. That is StrictMode telling you cleanup works.

Q12. Should a request ever continue after every component unmounts?

Sometimes yes: if the user will likely come back (tabs), finishing the request warms the cache. That is a policy choice. I abort to save bandwidth; React Query by default lets it finish. An option like `abortOnUnmount` would make it configurable.

Challenge 4: Dependencies and retry

Q13. Why keep the fetcher in a ref instead of the effect deps?

Callers usually pass an inline function, which is a new reference every render. In deps it would refetch every render, an infinite loop when the response triggers a render. The key identifies *what* to fetch; the fetcher is *how*, so only the key belongs in deps.

Q14. Should every error be retried?

No. 4xx errors (400, 401, 404) won't fix themselves, so retrying wastes time. Retry network errors and 5xx / 429. I'd make `retry` accept a function: `retry: (attempt, err) => err.status >= 500 && attempt < 3`. Also add jitter to the backoff so many clients don't retry in sync.

Implementation steps

Q15. Why is `mutate` overwritten by an in-flight refetch, and how would you fix it?

The refetch started before the mutation, so its response holds older server data and lands last. Fix for optimistic updates: `cancelQuery(key)` first, snapshot the old data, `setQueryData` the optimistic value, send the request, roll back to the snapshot on error, invalidate on success.

Q16. How does `keepPreviousData` work, and what is the risk?

The hook remembers the last non-undefined data in a ref and returns it while the new key has none, with `isPreviousData: true`. The risk is showing page 2's data labelled as page 3, so the UI dims it, disables Next, and shows an error instead of old data if the new page fails.

Q17. How would you test this?

The store is plain JS, so most logic is tested with node asserts (as in `check.mjs`): dedupe, retry count, abort, late-response race, shared-key release. For the hook: Vitest + React Testing Library, MSW for the network, fake timers for backoff, and a test that unmounts mid-request and asserts no warnings.

Performance and "what if"

Q18. What if the API is very slow?

Users see a skeleton on first load, cached data on revisits (`staleTime`), and previous data while paginating. I'd add a timeout via `AbortSignal.timeout(ms)` combined with the hook's signal, and maybe prefetch the next page on hover (`fetchQuery` without mounting).

Q19. Does one cache update re-render every component using `useFetch` ?

No. Listeners are stored per key, so `setEntry('/users/2')` only notifies hooks subscribed to `/users/2` . A global store with one listener set would re-render every consumer on every change.

Q20. What if the user goes offline?

Requests fail and retry with backoff; cached data stays on screen with a "refresh failed" hint. Next steps: listen to `online` / `offline` events, pause retries while offline, refetch active keys on reconnect, and optionally persist the cache to IndexedDB.

Trade-offs

Q21. Why a module singleton instead of a Context provider?

Fewer moving parts and it works outside React (prefetch from a router loader). The downside is global state: tests must reset it, and with SSR it would share data between users. For SSR I'd move the maps into a `QueryClient` object passed via Context.

Q22. How would you support Suspense?

In suspense mode, if the entry is loading, `throw` the in-flight promise; if it has an error, `throw` the error for an error boundary; otherwise return data. The dedupe map makes this safe because every suspended render throws the same promise.